

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Клиент-серверные приложения и сетевые технологии»

Пояснительная записка к курсовой работе

на тему:

«Разработка клиент-серверного приложения магазина игрушек artlo»

Выполнил: студент группы БИВТ-24-4

Кадыров Артур Рустамович

Подпись: _____

Проверили: Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Оглавление

1. Предметная область
2. Постановка задачи
3. Описание архитектуры
4. Описание структуры БД
5. Описание серверной части
6. Описание клиентской части
7. Заключение
8. Список литературы

1. Предметная область

Предметной областью курсовой работы является разработка персонального клиент-серверного веб-приложения магазина игрушек artlo. Приложение моделирует работу небольшой витрины игрушек: пользователь регистрируется, входит в систему, просматривает товары, добавляет их в корзину, оформляет заказ и видит изменение остатков.

В рамках предметной области выделены две основные группы сущностей: пользователи и товары. Пользователь содержит данные профиля и применяется для демонстрации обязательного API /users. Товар содержит название, категорию, цену, рейтинг, описание, изображение и остаток. Корзина хранится на стороне клиента и связывает пользователя с выбранными товарами.

Практическая проблема, решаемая приложением, состоит в организации удобного доступа к каталогу игрушек и в проверке корректности вводимых данных. Без серверной валидации пользователь мог бы создать профиль с пустым именем, неверным email или слишком коротким паролем. Без разграничения доступа любой пользователь мог бы удалять чужие профили. Поэтому в приложении реализованы регистрация, вход, роль администратора, обработка ошибок и клиентская админ-панель.

Тема выбрана так, чтобы наглядно показать изученные в курсе возможности: HTML/JSPX-разметку, CSS-оформление, REST API, LocalStorage, работу с HTTP-запросами и ответами, создание контроллеров и сервисов, использование DTO, декораторов, провайдеров, валидации, логирования и обработки исключений.

2. Постановка задачи

Цель работы — разработать и протестировать персональное клиент-серверное приложение artlo, оформить пояснительную записку по требованиям ГОСТ и подготовить исходный код к загрузке в LMS Moodle и GitHub-репозиторий.

По заданию API должен предоставлять обязательные endpoint: GET /users для получения списка пользователей, GET /users/:id для получения пользователя по идентификатору и POST /users для создания нового пользователя. Пользователь должен содержать id, name, email и age. Все поля, кроме age, являются обязательными, email должен быть валидным адресом электронной почты.

Данные пользователей по индивидуальной постановке должны храниться в памяти приложения и не сохраняться между перезапусками. Поэтому в проекте используется in-memoу хранение в сервисах NestJS. Это решение соответствует формулировке задания, хотя в общем PDF с требованиями к курсовым работам также описывается вариант с БД. В данной работе постоянная БД не используется именно из-за требования о хранении данных в памяти.

Для расширения функциональности и демонстрации темы дополнительно реализованы: регистрация с паролем, вход пользователя по email и паролю, вход администратора, удаление пользователей администратором, получение каталога игрушек, оформление заказа и уменьшение остатков товаров после заказа.

Итоговый проект должен демонстрировать HTML, CSS, Git, LocalStorage, API, управление доступом по категориям пользователей, использование библиотек для валидации данных, создание контроллеров и сервисов, декораторы и провайдеры NestJS, работу с HTTP-запросами и ответами, обработку ошибок и исключений.

3. Описание архитектуры

Приложение построено по клиент-серверной архитектуре. Клиентская часть расположена в папке Source/frontend и реализована на React + Vite. Серверная часть расположена в папке Source/backend и реализована на NestJS. Клиент и сервер запускаются отдельно: сервер на <http://localhost:3000>, клиент на адресе Vite, обычно <http://localhost:5173>.

React-клиент отвечает за интерфейс, регистрацию, вход, отображение товаров, корзину, админ-панель, хранение состояния в LocalStorage и отправку HTTP-запросов. NestJS-сервер отвечает за маршрутизацию запросов, валидацию DTO, работу сервисов, хранение данных в памяти, логирование, обработку исключений и возврат JSON-ответов.

Сервер разделен на модули. UsersModule отвечает за пользователей и обязательные endpoint /users. AuthModule отвечает за вход пользователя и администратора. ToysModule отвечает за каталог игрушек и оформление заказа. AppModule объединяет

эти модули в одно приложение.

Логика разделена по слоям: контроллеры принимают HTTP-запросы, сервисы выполняют прикладную логику, DTO описывают входные данные, а фильтр исключений формирует единый формат ошибок. Такой подход соответствует архитектурным практикам NestJS и упрощает защиту проекта: по каждому endpoint можно показать контроллер, сервис и DTO.

Компонент	Назначение
React + Vite	Клиентский интерфейс, регистрация, вход, каталог, корзина, админ-панель
NestJS API	REST API, валидация, сервисы, обработка ошибок, логирование
UsersModule	GET /users, GET /users/:id, POST /users, DELETE /users/:id
AuthModule	POST /auth/login для входа пользователя и администратора
ToysModule	GET /toys и POST /toys/checkout для каталога и заказа
LocalStorage	Хранение сессии и корзины на стороне клиента

Таблица 1 — 3. архитектуры

4. Описание структуры БД

В работе используется in-memory структура данных, а не физическая база данных. Это связано с прямым требованием индивидуального задания: данные должны храниться в памяти и не должны сохраняться между перезапусками приложения. Поэтому раздел описывает структуру данных, размещенных в сервисах NestJS.

Основная структура User используется для обязательных endpoint /users. Внутри сервера пользователь содержит id, name, email, passwordHash и age. Клиенту возвращается публичная версия пользователя без passwordHash. Такой подход позволяет показать безопасную работу с паролями: пароль не сохраняется открытым текстом и не возвращается через API.

Структура Toy используется для каталога магазина artlo. Она содержит id, title, category, price, available, stock, rating, description и image. В каталоге используется пять товаров с фотографиями: радиоуправляемая машина Subaru Drift 4WD, робот Vector, кукла Candy, пазл Planet Earth и плюшевый динозавр Rex.

Корзина хранится в LocalStorage на стороне клиента под ключом toy-shelf-cart. При оформлении заказа клиент отправляет товары на сервер через POST /toys/checkout. Сервер проверяет наличие товара и достаточный остаток, после чего уменьшает stock в массиве toys. После перезапуска сервера остатки возвращаются к исходным значениям, так как хранение является временным.

Сущность	Поля	Назначение
User	id, name, email, passwordHash, age	Профиль пользователя; passwordHash не возвращае
Toy	id, title, category, price, available, stock, link, description, category	Карточка товара, каталога
Cart	toyId, quantity	Корзина клиента в LocalStorage

Таблица 2 — 4. структуры БД

5. Описание серверной части

Серверная часть разработана на NestJS. В файле main.ts создается приложение через NestFactory, включается CORS, подключается глобальный ValidationPipe, регистрируется HttpExceptionHandler и RequestLoggerMiddleware. Сервер запускается на порту 3000.

UsersController реализует endpoint GET /users, GET /users/:id, POST /users и DELETE /users/:id. GET /users возвращает список публичных профилей. GET /users/:id ищет пользователя по id. POST /users создает нового пользователя после проверки DTO. DELETE /users/:id используется администратором для удаления пользователя.

UserService хранит массив пользователей в памяти. При создании пользователя сервис проверяет уникальность email, очищает имя от лишних пробелов, приводит email к нижнему регистру, генерирует id через randomUUID и сохраняет bcrypt-хеш пароля. Метод toPublicUser удаляет passwordHash из ответа API.

CreateUserDto задает правила валидации: name должен быть непустой строкой, email должен быть валидным адресом электронной почты, password должен иметь длину не меньше 6 символов, age является необязательным целым числом от 1 до 120. Для валидации используется class-validator, для преобразования age в число используется class-transformer.

AuthController содержит POST /auth/login. AuthService проверяет вход администратора по логину admin и паролю admin123, а обычного пользователя — по email и паролю. Для проверки пароля используется bcrypt.compareSync. В ответ возвращаются token, role, login и публичные данные пользователя.

ToysController содержит GET /toys и POST /toys/checkout. GET /toys возвращает каталог товаров. POST /toys/checkout принимает массив товаров из корзины, проверяет количество и остаток, уменьшает stock, обновляет available и возвращает сумму заказа вместе с обновленным каталогом.

HttpExceptionFilter формирует единый формат ошибок JSON: statusCode, timestamp и details. Это удобно для frontend, потому что все ошибки можно обработать одинаково. RequestLoggerMiddleware фиксирует метод, URL, статус ответа и время выполнения запроса, что помогает при тестировании и защите.

С точки зрения безопасности реализованы базовые учебные меры: пароль не хранится открытым текстом, passwordHash не отдается клиенту, email валидируется, лишние поля отбрасываются через ValidationPipe, а административная функция удаления пользователей показывается только после входа администратора. Для промышленного проекта можно дополнительно добавить JWT, guards и серверную проверку ролей на каждом защищенном endpoint.

Endpoint	Метод	Назначение
/users	GET	Получение списка пользователей
/users/:id	GET	Получение пользователя по id
/users	POST	Регистрация пользователя
/users/:id	DELETE	Удаление пользователя администратором
/auth/login	POST	Вход пользователя или администратора
/toys	GET	Получение каталога товаров
/toys/checkout	POST	Оформление заказа и уменьшение остатков

Таблица 3 — 5. серверной части

6. Описание клиентской части

Клиентская часть разработана на React + Vite. Основной файл App.jsx содержит состояние текущей сессии, формы входа, формы регистрации, списка пользователей, списка товаров, корзины, выбранной категории и сообщения об ошибке. API-запросы вынесены в api.js, работа с LocalStorage — в storage.js, оформление — в styles.css.

Стартовая страница содержит два режима: вход и регистрация. В режиме регистрации пользователь вводит имя, email, пароль и возраст. После отправки формы выполняется POST /users. В режиме входа пользователь выбирает тип входа: пользователь или администратор. Пользователь входит по email и паролю, администратор — по admin/admin123.

После входа открывается магазин artlo. В верхней части отображается бренд, название «Магазин игрушек» и карточка текущего пользователя. Ниже показана статистика: количество товаров, количество товаров в корзине и сумма заказа.

Каталог состоит из пяти карточек товаров с фотографиями, названием, категорией, рейтингом, описанием, ценой и остатком. Фильтр категорий позволяет показать только товары выбранной категории. Изображения товаров хранятся локально в Source/frontend/public/images, поэтому сайт не зависит от внешних картинок.

Корзина находится справа от каталога. Пользователь может добавлять товары, уменьшать и увеличивать количество, удалять позицию из корзины и оформить заказ.

Корзина сохраняется в LocalStorage, поэтому выбранные товары не исчезают после обновления страницы. При оформлении заказа frontend отправляет POST /toys/checkout, очищает корзину и обновляет остатки товаров.

Администратор после входа видит блок «Управление пользователями». В нем выводится список пользователей и кнопки удаления. При нажатии на кнопку выполняется DELETE /users/:id, после чего список обновляется через GET /users.

CSS-оформление сделано так, чтобы сайт не выглядел стандартной белой страницей: используется цветной фон, темная брендовая стартовая панель, акцентные кнопки, карточки с тенями, цветовые бейджи остатков и адаптивная сетка. Фотография куклы отображается через object-fit: contain, поэтому вертикальное изображение не обрезается.

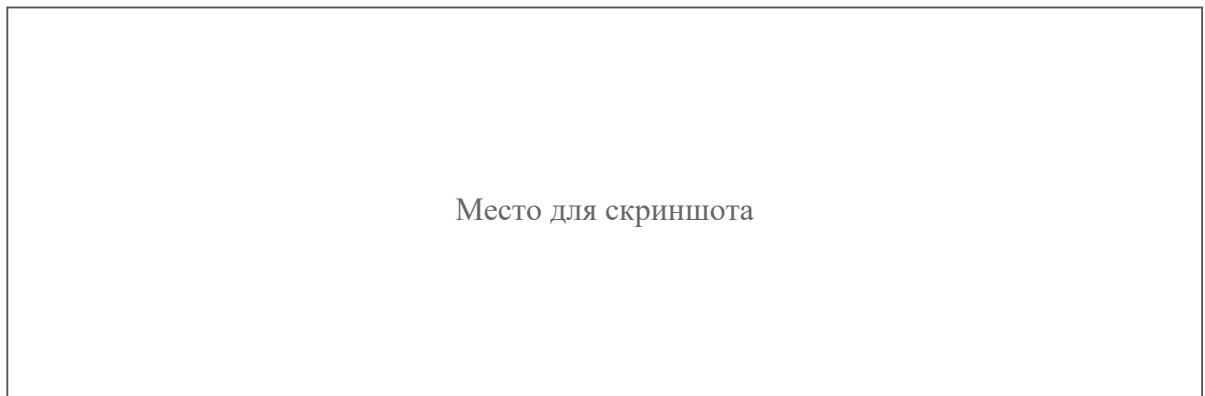


Рисунок 1 — Стартовая страница входа и регистрации

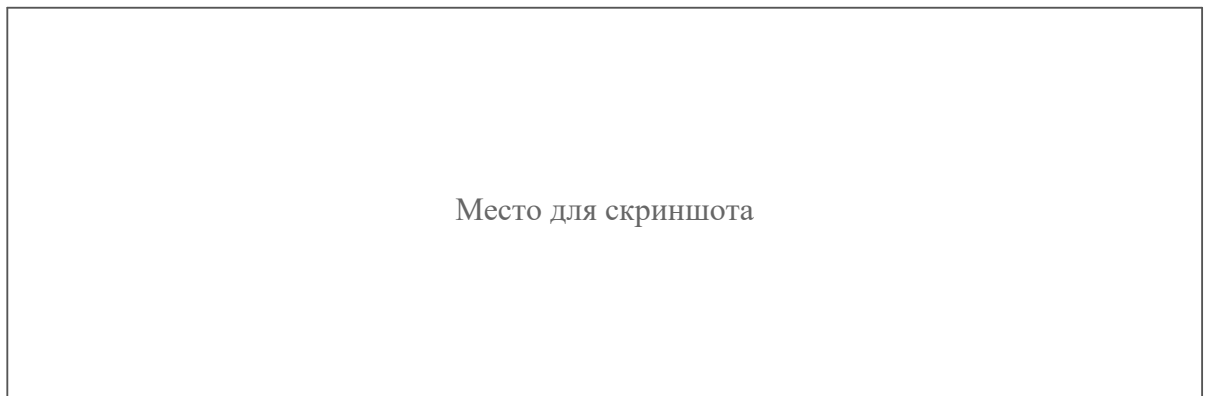


Рисунок 2 — Регистрация пользователя с паролем

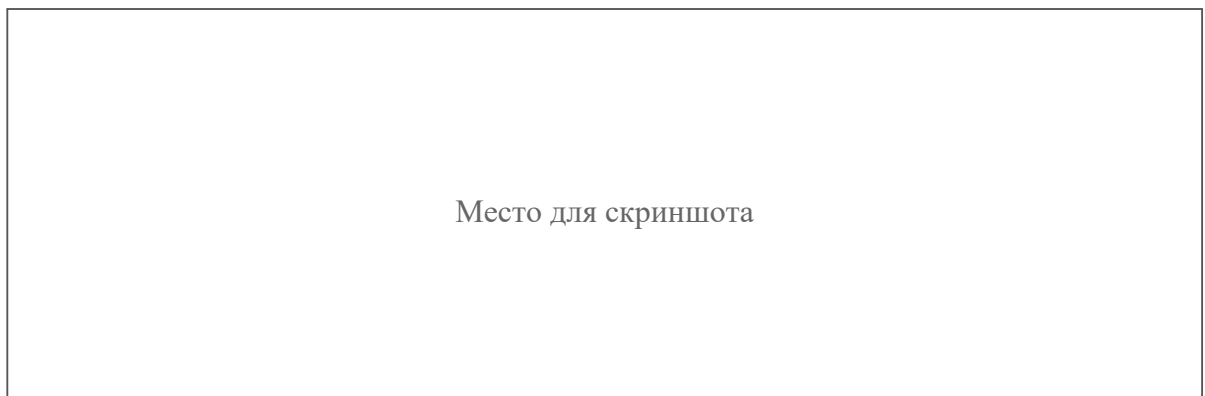


Рисунок 3 — Вход администратора

Место для скриншота

Рисунок 4 — Главная страница магазина artlo

Место для скриншота

Рисунок 5 — Каталог товаров с фотографиями

Место для скриншота

Рисунок 6 — Корзина и оформление заказа

Место для скриншота

Рисунок 7 — Изменение остатка товара после заказа



Рисунок 8 — Админ-панель удаления пользователей

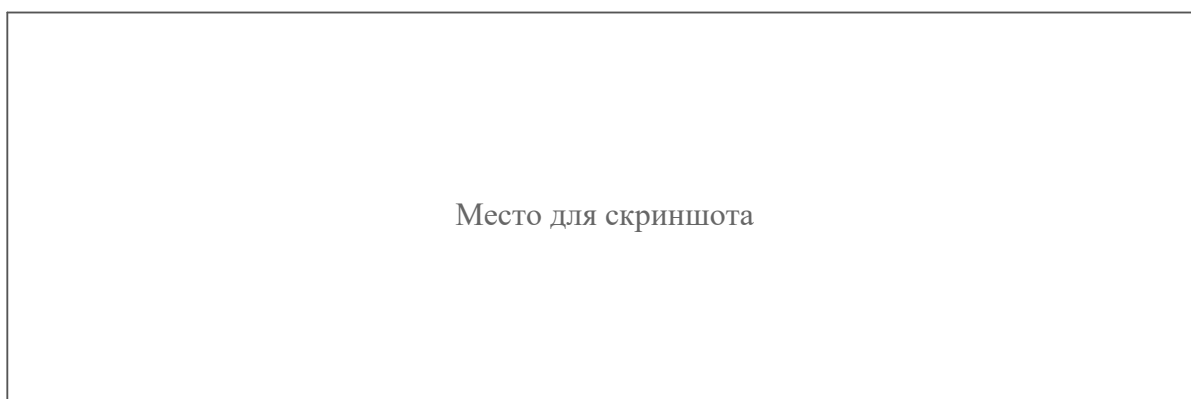


Рисунок 9 — Ответ GET /users или GET /toys



Рисунок 10 — Логи HTTP-запросов backend

7. Заключение

В результате выполнения курсовой работы разработано клиент-серверное приложение artlo. Приложение реализует обязательные endpoint /users, хранит данные в памяти, проверяет обязательные поля, валидирует email, использует пароль при регистрации и поддерживает вход пользователя.

Серверная часть демонстрирует NestJS-модули, контроллеры, сервисы, DTO, декораторы, провайдеры, ValidationPipe, class-validator, class-transformer, bcryptjs, обработку исключений и логирование запросов. Клиентская часть демонстрирует HTML/JSX, CSS, Fetch API, LocalStorage, регистрацию, вход, корзину, оформление

заказа, обновление остатков и админское удаление пользователей.

Работа приложения проверяется через браузер и HTTP-запросы. Основные сценарии: регистрация пользователя, вход пользователя, вход администратора, получение списка пользователей, удаление пользователя администратором, получение каталога товаров, добавление товара в корзину, оформление заказа и уменьшение остатка.

Проект опубликован в GitHub-репозитории. Для запуска необходимо установить Node.js, выполнить `npm.cmd install` в backend и frontend, затем запустить backend через `npm.cmd run start:dev` и frontend через `npm.cmd run dev`.

8. Список литературы

1. NestJS Documentation [Электронный ресурс]. URL: <https://docs.nestjs.com/> (дата обращения: 07.06.2026).
2. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 07.06.2026).
3. Vite Documentation [Электронный ресурс]. URL: <https://vite.dev/> (дата обращения: 07.06.2026).
4. MDN Web Docs. Fetch API [Электронный ресурс]. URL: https://developer.mozilla.org/docs/Web/API/Fetch_API (дата обращения: 07.06.2026).
5. MDN Web Docs. Window: localStorage property [Электронный ресурс]. URL: <https://developer.mozilla.org/docs/Web/API/Window/localStorage> (дата обращения: 07.06.2026).
6. class-validator. GitHub repository [Электронный ресурс]. URL: <https://github.com/typestack/class-validator> (дата обращения: 07.06.2026).
7. bcryptjs. npm package [Электронный ресурс]. URL: <https://www.npmjs.com/package/bcryptjs> (дата обращения: 07.06.2026).
8. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.
9. Репозиторий исходного кода приложения artlo [Электронный ресурс]. URL: <https://github.com/Artlogg/toy-shelf-coursework> (дата обращения: 07.06.2026).